

“사각형 면적” 문제 풀이

작성자: 박현민

부분문제 1

예제로 주어지는 두 데이터가 전부인 부분문제이다. 입력을 받고 특성에 따라 이미 구해진 답을 출력만 하면 된다.

부분문제 2

자르는 횟수가 1회이므로 후처리를 생각하지 않고 풀 수 있다. 가로로 자르고 남은 위쪽 부분의 면적과 세로로 자르고 남은 왼쪽 부분의 면적을 비교하여 더 크거나 같은 값이 답이다.

부분문제 3

항상 가로로 자르고 남은 위쪽 부분의 면적이 세로로 자르고 남은 왼쪽 부분의 면적보다 크거나 같음이 보장되므로, 언제나 가로로 자르기만 한다는 것도 보장된다. 이에 매 자르는 시행마다 주어진 좌표 (X, Y) 에 따라 만약 무시되지 않는 점이라면, 현재 남은 종이의 세로 길이인 A 의 값을 X 로 바꿔준다.

모든 시행을 끝내고 나면 남은 종이의 세로 길이 A 와 가로 길이 B 를 곱하여 답을 얻을 수 있다.

부분문제 4

매 자르는 시행마다 주어진 좌표 (X, Y) 에 따라 만약 무시되지 않는 점이라면, 다음 두 가지 경우가 있다.

1. 가로로 자르고 남은 위쪽 부분의 면적이 세로로 자르고 남은 왼쪽 부분의 면적보다 크거나 같은 경우
→ 현재 남은 종이의 세로 길이인 A 의 값을 X 로 바꿔준다.
2. 가로로 자르고 남은 위쪽 부분의 면적이 세로로 자르고 남은 왼쪽 부분의 면적보다 작은 경우 → 현재 남은 종이의 가로 길이인 B 의 값을 Y 로 바꿔준다.

모든 시행을 끝내고 나면 남은 종이의 세로 길이 A 와 가로 길이 B 를 곱하여 답을 얻을 수 있다.

“계산 로봇” 문제 풀이

작성자: 박현민

부분문제 1

문제의 정의상 제일 왼쪽 열에 있는 로봇의 입력 값은 0 하나인데, 열의 개수가 1이므로 모든 로봇의 저장 값도 0이다. 따라서 답은 항상 0이다.

부분문제 2

부분문제 1에 이어, 제일 왼쪽 열에 있는 로봇의 출력 값은 모든 $i(1 \leq i \leq M)$ 에 대해 $D_{i,1}$ 임을 알 수 있다. 그렇다면 2번째 열의 저장 값은 어떻게 정해지는지 살펴보자.

- 좌표 $(1, 2)$ 에 있는 로봇의 저장 값: $D_{1,1}, D_{2,1}$ 중 최댓값
- 좌표 $(2, 2)$ 에 있는 로봇의 저장 값: $D_{1,1}, D_{2,1}, D_{3,1}$ 중 최댓값
- 좌표 $(3, 2)$ 에 있는 로봇의 저장 값: $D_{2,1}, D_{3,1}, D_{4,1}$ 중 최댓값
- 좌표 $(4, 2)$ 에 있는 로봇의 저장 값: $D_{3,1}, D_{4,1}, D_{5,1}$ 중 최댓값
- ...
- 좌표 $(M-1, 2)$ 에 있는 로봇의 저장 값: $D_{M-2,1}, D_{M-1,1}, D_{M,1}$ 중 최댓값
- 좌표 $(M, 2)$ 에 있는 로봇의 저장 값: $D_{M-1,1}, D_{M,1}$ 중 최댓값

부분문제 1에서 제일 왼쪽 열에 있는 모든 로봇의 저장 값은 0임을 보았다. 따라서 이 부분문제의 답은 $i(1 \leq i \leq M)$ 에 대해 좌표 $(i, 2)$ 에 있는 로봇들의 저장 값 중 최댓값과 같고, 이는 위에서 살펴본 것에 따라 $D_{i,1}$ 중 최댓값과도 같다.

부분문제 3

왼쪽부터 각 로봇의 저장 값과 출력 값이 어떻게 정해지는지 살펴보자.

- 좌표 $(1, 1)$ 에 있는 로봇: 저장 값은 문제의 정의에 의해 0이고, 출력 값은 $D_{1,1}$ 이다.
- 좌표 $(1, 2)$ 에 있는 로봇: 저장 값은 $D_{1,1}$, 출력 값은 $D_{1,1} + D_{1,2}$ 이다.
- 좌표 $(1, 3)$ 에 있는 로봇: 저장 값은 $D_{1,1}, (D_{1,1} + D_{1,2})$ 중 최댓값인데, 모든 가중치는 양수이므로 $D_{1,1} + D_{1,2}$ 이다. 따라서 출력 값은 $D_{1,1} + D_{1,2} + D_{1,3}$ 이다.
- 좌표 $(1, 4)$ 에 있는 로봇: 저장 값은 $D_{1,1}, (D_{1,1} + D_{1,2}), (D_{1,1} + D_{1,2} + D_{1,3})$ 중 최댓값인데, 모든 가중치는 양수이므로 가장 마지막 값이다. 출력 값은 역시 이 값에 $D_{1,4}$ 를 더한 값이다.
- ...

마지막으로 좌표 $(1, M)$ 에 있는 로봇의 저장 값은 $D_{1,1}, (D_{1,1} + D_{1,2}), (D_{1,1} + D_{1,2} + D_{1,3}), \dots, (D_{1,1} + D_{1,2} + \dots + D_{1,M-2}), (D_{1,1} + D_{1,2} + \dots + D_{1,M-1})$ 중 최댓값이고 이번에도 역시 가장 마지막 값이다. 답인 로봇들의 저장 값 중 최댓값도 이 값이다.

부분문제 4

문제에서 주어진 순서대로 $j(1 \leq j \leq N)$ 가 증가하는 방향으로 살펴본다. 각 좌표에 있는 로봇의 입력 값들이 모두 왼쪽에 주어졌고 이를 순회하며 최댓값을 찾아 저장 값으로 가져오는 과정은 $O(MN)$ 에 할 수 있다. 로봇이 총 MN 개 있으므로 전체 시간복잡도는 $O(M^2N^2)$ 이고, 이 부분문제의 제약조건 상 충분히 시간 내에 동작한다.

부분문제 5

각 로봇의 입력 값들을 관찰하면 영역이 중복된다는 점에서 다음과 같은 성질을 찾을 수 있다.

$1 \leq i \leq M, 1 < j \leq N$ 를 만족하는 모든 i, j 에 대해 좌표 (i, j) 에 있는 로봇의 저장 값은 다음 값들 중 최댓값이다:

- ($i > 1$ 인 경우) 좌표 $(i - 1, j)$ 에 있는 로봇의 출력 값
- 좌표 (i, j) 에 있는 로봇의 출력 값
- ($i < M$ 인 경우) 좌표 $(i + 1, j)$ 에 있는 로봇의 출력 값

위와 같이 계산하면 시간복잡도 $O(MN)$ 만에 문제를 해결할 수 있다.

“괄호의 값 비교” 문제 풀이

작성자: 김준겸

부분문제 1

길이가 6 이하인 괄호열의 개수는 8개로, 나열하면 $()$, $(())$, $()()$, $()()()$, $(())()$, $(())()$, $()()$, $()()()$ 이다. 따라서 이 모든 괄호열에 대해 손으로 괄호값을 계산한 후 이를 프로그램에 직접 작성하여 값을 비교할 수 있다. 참고로, 이 중 $(())()$ 을 제외한 7개의 괄호열의 경우 문제 설명에서 괄호값을 계산해주었다.

부분문제 2

각 괄호열의 길이가 50 이하이므로, 괄호값이 최대가 되는 경우는 (25개가 온 뒤) 25개가 오는 경우이다. 이때 괄호값은 2^{24} 로, 32비트 정수 범위 안이다. 따라서 어떠한 방법으로든 괄호값을 직접 계산하여 문제를 해결할 수 있다.

일반적인 방법은 앞에서부터 괄호를 보며 중첩된 횟수를 바탕으로 괄호값을 계산하는 것이다. 괄호열의 괄호값은 모든 부분 괄호열 $()$ 에 괄호가 씌워진 횟수만큼 2가 거듭해 곱해진 값을 모두 더함으로써 계산된다. 주어진 괄호열이 올바른 괄호열임이 보장되므로, 부분 괄호열 $()$ 가 있을 때 이 괄호열은 최종적으로 앞에 있는 여는 괄호의 개수에서 닫는 괄호의 개수를 뺀 것 만큼 거듭제곱한 값을 가진다. 이를 매번 계산하면 괄호열 길이 N 에 대해 $O(N^3)$ 이고, 2의 거듭제곱을 미리 계산해두거나 비트 연산을 사용하면 $O(N^2)$ 이다. 이에 더해 부분 합을 사용하면 $O(N)$ 에 해결할 수 있다. 스택을 사용하거나, 중첩된 횟수를 관리하여 앞에서부터 괄호열을 훑으며 여는 괄호를 만나면 중첩된 횟수를 하나 증가시키고, 닫는 괄호를 만나면 중첩된 횟수를 하나 감소시키는 방법(스위핑 기법)으로도 $O(N)$ 에 해결할 수 있다.

제한이 상당히 작으므로, 재귀함수나 언어 내장 문자열 처리 함수를 이용해서 문제에서 제공한 정의에 따라 직접 파싱을 시도할 수도 있다. 보통 이 경우 시간복잡도는 $O(N^2)$ 이다.

부분문제 3

괄호열의 길이가 길므로 이 부분문제부터는 괄호열을 직접 계산하려고 시도하면 오버플로우가 발생할 수 있다. 단순 괄호열의 성질을 이용하면, 괄호값을 십진수의 형태로 명시적으로 계산하지 않아도 두 괄호열을 비교할 수 있다. 길이가 $2N$ 인 단순 괄호열의 괄호값은 2^{N-1} 이고, 이어붙인 모든 단순 괄호열의 괄호값이 다르므로 각 괄호열의 괄호값은 서로 다른 2의 거듭제곱의 합 꼴로 나타난다.

$2^0 + 2^1 + \dots + 2^{n-1} < 2^n$ 이므로, 두 괄호값에서 나타나는 가장 큰 2의 거듭제곱 꼴이 다르다면 큰 쪽이 반드시 크다. 만약 가장 큰 2의 거듭제곱 꼴이 같다면, 둘 다 소거하고 남은 거듭제곱 꼴끼리 비교할 수 있다. 모든 거듭제곱 꼴이 같다면 두 괄호열이 같다. 시간복잡도는 괄호열 길이의 합을 N 이라고 할 때 $O(N)$ 이다.

위 풀이를 심화된 관점에서 보면, 각 괄호값을 이진수로 나타낼 수 있다. 서로 다른 2의 거듭제곱의 합은 2^i 가 존재할 경우 뒤에서 i 번째 자리를 1, 존재하지 않을 경우 0으로 두면 이진수로 나타나므로 두 이진수를 비교하는 것과 같아진다.

별해로, 큰 수 자료형을 매우 적절히 사용하면 오버플로우를 내지 않고 위 서브태스크를 계산할 수 있다. 모든 단순 괄호열의 길이가 다르므로 일어나는 덧셈의 횟수는 많아야 괄호열의 길이 N 에 대해 $O(\sqrt{N})$ 이다. 큰 수 자료형에서 덧셈 연산을 수행할 때의 시간복잡도는 수의 크기 X 에 대해 $O(\lg X)$ 이고, 거듭제곱의 경우 단순하게 계산할 경우 필요한 계산량은 매우 많지만 계산해야 하는 값이 모두 2의 거듭제곱 꼴이라는 점을 이용해서 연산에 필요한 비용을 크게 낮출 수 있다. 위와 같은 최적화를 실제로 구현하거나, 언어에서

기본적으로 제공하는 기능을 잘 사용하면 부분문제 3을 부분문제 2의 $O(N)$ 풀이와 동일한 해법을 이용해 해결할 수 있다.

부분문제 4

부분문제 2에서 본 것과 같이, 괄호값은 2의 거듭제곱의 합 꼴로 나타난다. 단 부분문제 3과 같이 서로 다른 거듭제곱의 합 꼴이 아니므로, 부분문제 3과 같이 단순히 해결할 수는 없다.

먼저 괄호값을 부분문제 3과 같이 2의 거듭제곱의 합 꼴로 나타내되, 부분문제 2에서 사용한 $O(N)$ 계산 방식을 사용하여야 한다. 문제는 $2^x + 2^x = 2^{x+1}$ 과 같이, 괄호값을 구성하는 각 2의 거듭제곱들이 달라도 실제 값은 같을 수 있다는 점이다. 따라서 두 괄호값을 비교하기 위해서는 기존 거듭제곱의 합 꼴을 비교에 더 용이한 꼴로 바꿀 필요가 있다.

만약 한 괄호열의 괄호값을 2의 거듭제곱 꼴로 나타내었을 때 2^x 가 합에 존재할 경우 유일하다고 가정하자. 이 경우 부분문제 3의 경우와 같으므로 같은 방법을 사용하여 두 괄호열의 값을 비교할 수 있다. 이 꼴은 2^i 가 합에 2개 이상 존재할 때, 2^i 가 0개 또는 1개만 남을 때까지 2개씩 계속 합쳐 2^{i+1} 로 만드는 것으로 수행할 수 있다. 만약 i 를 0부터 충분히 큰 수, 이 문제에서는 괄호열의 길이 $2N$ 에 대해 $N - 1$ 까지로 하여 위 작업을 반복한다면 $O(N)$ 에 전체 과정을 완료할 수 있다. 따라서 문제를 해결할 수 있다.

역시 위 풀이를 심화된 관점에서 보면, 이진수의 덧셈을 받아올림 하는 것으로 간주할 수 있다. $2^i + 2^i = 2^{i+1}$ 을 $10 \dots 00_{(2)} + 10 \dots 00_{(2)} = 100 \dots 00_{(2)}$ 로 나타낼 수 있는 점에 주목하자. 어떤 수를 이진수로 표현하는 방식은 유일하므로, 각 괄호열을 이진수로 나타낸 다음 비교한다고 생각할 수 있다. 다수의 십진수를 더한 다음 받아올림할 때처럼, 특정 자리수를 모두 더한 뒤 2 이상이라면 2로 나눈 나머지를 쓰고 나눈 몫을 받아올림하면 계산 가능하다.

“그래프 균형 맞추기” 문제 풀이

작성자: 김준원

부분문제 1

아주 간단한 형태의 그래프이므로 직접 예제 그림을 그려 생각할 수 있다. 간선의 가중치 조건이 입력으로 주어졌을 때, 정점의 가중치로 가능한 수는 유일하게 정해진다. 일반적으로

- 1번 정점과 2번 정점을 잇는 간선의 가중치가 c_{1-2}
- 2번 정점과 3번 정점을 잇는 간선의 가중치가 c_{2-3}
- 3번 정점과 1번 정점을 잇는 간선의 가중치가 c_{3-1}

로 주어지면, 정점에 부여할 가중치는 1번 정점에 $(c_{1-2} + c_{3-1} - c_{2-3})/2$, 2번 정점에 $(c_{2-3} + c_{1-2} - c_{3-1})/2$, 3번 정점에 $(c_{3-1} + c_{2-3} - c_{1-2})/2$ 으로 유일하게 정해진다. 이 값들을 직접 계산해서 출력하면 된다. 가중치는 정수여야만 하므로, 이 값들이 정수가 아니라면, No를 출력해야 함에 주의하라.

부분문제 2, 3, 4

부분문제 2, 3에서 입력 그래프는 직선이다. 여기서 i 번 정점과 $i+1$ 번 정점을 잇고 있는 간선의 가중치를 c_i 라고 하자.

- 만약 1번 정점에 가중치 x 를 부여했다면,
- 2번 정점에 가중치 $c_1 - x$ 를 부여해야 하고,
- 3번 정점에 가중치 $c_2 - c_1 + x$ 를 부여해야 하고,
- 4번 정점에 가중치 $c_3 - c_2 + c_1 - x$ 를 부여해야 하고,
- ...

와 같이, 한 정점에 부여할 가중치가 정해지면, 다른 각 정점에 부여할 가중치가 정해진다. 이 성질은 부분문제 4의 제약 조건을 만족하는 그래프(이런 그래프를 ‘트리’라고 부른다)에서도 마찬가지라서, 1번 정점의 가중치가 x 라고 할 때, 다른 각 정점 i 의 가중치는 $x + k_i$ 또는 $-(x + k_i)$ (k_i 는 상수)의 꼴로 정해진다. 이들 k_i 들은 부분문제 2, 3에서는 반복문을, 부분문제 4에서는 깊이 우선 탐색(DFS)을 이용해 구할 수 있다.

부분문제 1에서는 정점의 가중치가 유일하게 정해졌지만, 부분문제 2, 3, 4에서는 정점의 가중치를 부여하는 방법이 유일하지 않다. 1번 정점의 가중치 x 를 바꾸면, 다른 모든 정점의 가중치를 바꿀 수 있다. 반대로, 문제의 조건을 만족하도록 정점에 가중치를 부여하면, 이 가중치들은 모두 x 에 대한 식으로 쓸 수 있다.

이제 우리는 부분문제 2, 3, 4에서 정점에 가중치를 부여하는 방법은 무조건 존재한다(게다가, 무한정 많이 존재한다)는 사실을 안다. 이들 가중치를 모두 x 에 대한 식으로 쓸 수 있으므로, 가중치를 부여하는 비용 또한 아래와 같이 x 에 대한 식 $f(x)$ 로 쓸 수 있다.

$$f(x) = |x + k_1| + |x + k_2| + \cdots + |x + k_N|$$

함수 $f(x)$ 는 아래로 볼록한 꺾은선 모양으로 나타남을 관찰할 수 있다. 이 사실에서 x 가 N 개의 수 $-k_1, -k_2, \dots, -k_N$ 중 하나일 때 최솟값을 가진다는 사실을 알 수 있다. 이들 후보들을 모두 직접 시험해서 가장 작은 비용을 출력하면 $O(N^2)$ 에 부분문제 2를 풀 수 있다.

그리고 추가적인 관찰을 통해 x 가 N 개의 수 $-k_1, -k_2, \dots, -k_N$ 의 중간값일 때에 최솟값을 가진다는 사실을 알 수 있다. 이를 이용해 최적의 x 를 바로 알 수 있고, $O(N \log N)$ 에 부분문제 3, 4를 풀 수 있다.

부분문제 5

부분문제 5에서 입력 그래프는 목걸이 모양이다(즉, 하나의 ‘사이클’로 이루어져 있다). 즉 그래프는, 아래 그래프에서 정점과 간선의 순서를 임의로 섞은 꼴이다.

- i ($1 \leq i \leq N-1$)번째 간선은 i 번 정점과 $i+1$ 번 정점을 잇고 있으며, 가중치는 c_i 이다.
- N 번째 간선은 N 번 정점과 1번 정점을 잇고 있으며, 가중치는 c_N 이다.

정점의 순서를 섞어도 본질은 변하지 않기 때문에, 위와 같은 그래프가 입력으로 주어졌다고 하자.

부분문제 2, 3, 4의 아이디어와 같이 1번 정점에 x 의 가중치를 부여하고 나머지 정점에 부여할 가중치를 x 에 대한 식으로 나타낸다. 마지막 N 번째 간선을 무시하면 부분문제 2, 3에서 등장했던 직선형 그래프가 된다. 따라서 각 정점의 가중치를 x 에 대한 식으로 나타낼 수 있다. 하지만 N 번째 간선의 존재 때문에 확인할 게 하나 더 생긴다. 그래프의 크기(사이클의 크기)의 홀짝성에 따라 두 가지 경우로 나눌 수 있다.

사이클의 크기가 짝수일 때, N 번 정점은 $-(x + k_N)$ 꼴의 가중치를 가지게 된다. N 번째 간선에 의해 $x + -(x + k_N) = c_N$ 이라는 조건이 추가된다.

- 만약 $k_N \neq c_N$ 이라면 그래프의 균형이 맞도록 가중치를 부여할 수 있는 방법이 없고,
- 그렇지 않다면 N 번째 간선은 무시할 수 있다.

사이클의 크기가 홀수일 때, N 번 정점은 $x + k_N$ 꼴의 가중치를 가지게 된다. N 번째 간선에 의해 $x + (x + k_N) = c_N$ 이라는 조건이 추가된다. 이 조건에 의해 $x = (c_N - k_N)/2$ 가 되어야만 한다.

- 만약 x 가 정수가 아니라면, 그래프의 균형이 맞도록 정수 가중치를 부여할 수 있는 방법이 없고,
- x 가 정수라면, $x = (c_N - k_N)/2$ 를 각 식에 대입했을 때의 가중치가 유일한 답이 된다.

부분문제 6, 7

부분문제 2, 3, 4와 부분문제 5를 풀었다면, 전체 문제를 푼 것이나 다름없다. 풀이의 줄거리는 다음과 같다.

1번 정점에 가중치 x 를 부여했을 때, 다른 모든 정점 i 에 부여할 가중치가 간선을 따라 $x + k_i$ 또는 $-(x + k_i)$ 의 꼴로 정해진다. 하지만 그래프에 사이클이 있다면, 입력 조건들 사이 ‘충돌’이 발생할 수 있다. 충돌은 부분문제 5와 같이 경우를 나누어 해결해줄 수 있다. 충돌을 처리한 결과, i) 불가능함이 증명되거나, ii) x 의 값이 유일하게 정해지거나, iii) x 의 값이 정해지지 않을 수 있다. i), ii)의 경우에는 해당하는 답을 바로 출력할 수 있고, iii)의 경우에는 부분문제 2, 3, 4의 풀이와 같이 x 를 $-k_1, \dots, -k_N$ 의 중간값으로 두어 비용을 최소화할 수 있다.